

Recombination in a Self-Modifying Network Search: Building Blocks, and the Weight Inheritance That Crossover Needs

Vasili Gavrilov*

2026

Abstract

Ordinarily a neural network must be handed its shape before it can learn: how many layers, how wide, which activation. The system studied here is handed only the training data and a target error, and finds the rest itself by evolution: its depth and width, whether each layer is dense or a weight-sharing convolutional one, and its own learning rate, momentum, and activation. Onto that mutation-driven search we add *crossover*, recombination of two parents, following the intuition that mixing building blocks should reach a good architecture faster than mutating one lineage. The paper follows that intuition to a resolved conclusion, in three experiments. First, on a problem with genuine separable building blocks, concatenated deceptive trap functions, our one-point crossover beats mutation decisively, its advantage *growing* with the number of blocks, the textbook fingerprint of recombination. Second, on the neural architecture search, crossover is on-par with mutation, no better; we diagnose why, with measurements, and the reason is structural: neural capacity is fungible, and because each candidate is trained from scratch, the genome carries architecture but not the learned solution, so there is nothing for recombination to transfer. Third, we supply the missing ingredient, *weight inheritance*: a candidate is a modular set of sub-networks that carry their trained weights across generations, so a solved part is a self-contained transferable block. Now crossover wins again, and by a wide margin: on a task of several hard components it reaches the solution in about twelve generations regardless of component count, while mutation is several times slower and often fails outright. The building-block intuition holds for neural architecture search exactly when the genome carries the solution, not only its shape.

1 Introduction

Neural architecture search (NAS) automates a choice a human normally makes: the shape of the network. Its methods differ mainly in the search strategy: reinforcement learning [8], gradient-based relaxation [10], Bayesian optimization, random search, and evolutionary search [12]. Ours is evolutionary: the topology is evolved while each candidate's weights are trained by back-propagation [9]. Ordinarily the architecture and training knobs are arguments the user supplies, `predict(topology, activation, ...)`; the system here is handed only the data and a target error, `self_modifying_predict(data, error)`, and discovers the rest. The engine grew out of the author's 1997 seminar thesis, which derived the general multi-layer delta rule the training still uses [2].

A recurring result in this field is that *random search is hard to beat* [7, 11]. This paper asks whether *recombination* beats mutation at architecture search, and reaches a two-sided answer

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including rule engines and expert systems. Reference implementation: <https://github.com/Anode1/SMBPANN>.

that resolves cleanly: crossover needs building blocks that carry a solution, standard NAS does not provide them, and adding weight inheritance does. The self-adaptive machinery and the evolution-strategy view we lean on are standard in the optimization literature the author has worked in [3].

2 Method

2.1 The self-modifying search

A network is a feed-forward stack of dense and convolution-like layers. For a dense layer ℓ the forward pass is $a^{(\ell)} = g(W^{(\ell)}a^{(\ell-1)} + b^{(\ell)})$, with g a per-network hidden activation (logistic sigmoid, tanh, or ReLU) chosen by the genome; the output layer is a dense sigmoid. A convolutional layer applies F shared kernels of width K across the previous layer, so its width is derived, $d_\ell = F(d_{\ell-1} - K + 1)$: the local, weight-sharing structure LeCun introduced for images [6], here a move the search can take. Weights are trained by back-propagation with momentum (the multi-layer delta rule of the 1997 thesis [2]), initialized uniformly in $[-2.4/f_i, 2.4/f_i]$. All arithmetic is 32-bit float; all randomness flows through a seeded xorshift generator.

A *genome* carries the topology (per hidden layer: dense or convolutional, and its free parameters) and its own training hyper-parameters (learning rate, momentum, activation), plus a self-adaptive mutation rate. One *mutation* makes a single small change: perturb a layer, flip a layer dense $\leftrightarrow$ conv, or add or remove a layer. Reproduction is self-adaptive in the sense of evolution strategies [3]. The search keeps a population, retains the best few (elitism), and refills by reproduction, against a matched random-search control.

2.2 The crossover operator

To the mutation-only search we add optional *crossover* (**evolve -X**): a child is built from two elite parents by a one-point splice of their hidden-layer lists, a prefix of one and a suffix of the other, with a blend of their hyper-parameters, then mutated as usual. With **-X 0** the operator is inert and the run is byte-identical to the mutation-only search.

3 Experiments

3.1 The operator works: a building-block validation

We first check that the operator does what crossover should, on a problem built to have separable building blocks: concatenated deceptive trap functions, a genome of M blocks of K bits, where a block scores K only if all its bits are one, else $K - 1$ minus its number of ones, a deceptive gradient luring a bit-flipping search toward all-zero. The global optimum (all ones) is reachable only by assembling correctly-solved blocks. We race the same GA (tournament selection, elitism) with one-point crossover against mutation-only, over 200 seeds, scanning the block count (**make bbtest**).

Table 1 is decisive: at sixteen blocks mutation reaches the optimum on 28 of 200 seeds, crossover on 199. The operator and methodology are sound; when separable building blocks exist, crossover finds and combines them, and its advantage scales with the amount of structure.

3.2 On the architecture search, crossover does not help

We then run the same on/off comparison on the architecture search: for each seed, a fresh synthetic classification task ($d = 8$ inputs, a random-projection sine label), the self-modifying search run twice from the identical initial population, once mutation-only and once with crossover, each until its best validation error reaches a target. Over the paired runs, crossover and mutation

Table 1: Crossover versus mutation on concatenated deceptive traps ($K = 4$ -bit blocks, population 1000, tournament 3, 200 seeds). As the number of building blocks grows, crossover’s advantage grows: mutation’s cost explodes and it begins to fail while crossover keeps solving.

blocks M (genome)	mutation-only	crossover + mutation	speedup
4 (16 bits)	9.2 gens, 200/200	5.1 gens, 200/200	1.8×
8 (32 bits)	70.8 gens, 200/200	14.2 gens, 200/200	5×
12 (48 bits)	257 gens, 185/200	22.6 gens, 200/200	11×, more reliable
16 (64 bits)	363 gens, 28/200	42.3 gens, 199/200	mutation fails

are indistinguishable: the same fraction reached the target, the same median generations, an even head-to-head. Adding recombination did nothing, and not because the operator is weak (Table 1).

3.3 Why: the genome carries architecture, not the solution

Two measurements explain it. First, architecture alone does not solve the task: trained with default hyper-parameters, every topology, dense or convolutional, scored 0.25–0.59 test-MSE, at or worse than chance. The *hyper-parameters* do the work, brittly: holding a small 8, 4, 1 topology and varying only the learning rate, momentum, and activation swings the error from 0.09 to 0.65; with a good hyper-parameter set fixed, varying the topology is narrow and non-monotone (8, 4, 1 $\rightarrow$ 0.09 while 8, 8, 1 $\rightarrow$ 0.37). The two dimensions are entangled and the landscape rugged, and crossover of a topology block breaks the co-adaptation that made it good. Second, a parallel-branch probe (`make modnas`, gradient-checked) shows that even a purpose-built modular architecture does not decompose, because *neural capacity is fungible*: one sufficiently-wide branch does what several specialised ones would, so there are no independent modules to recombine.

Underneath both is one fact. In the trap problem the bits *were* the solution, so crossover transferred solutions. In architecture search the genome is architecture and the solution is *weights*, retrained from scratch for every candidate, so the genome carries no solution to transfer. There is nothing for recombination to recombine.

3.4 Weight inheritance restores the building blocks

That diagnosis names its own fix: let the genome carry the learned state, not only the shape. We test it in a modular setting (`make modevo`, back-propagation gradient-checked). A candidate is G independent sub-networks, one per output component; each component is a hard, high-frequency sine of its own input group, so training a sub-network from a random start succeeds only sometimes, a lottery. A candidate *carries its sub-networks’ trained weights across generations* and fine-tunes them further, so a solved component is a self-contained, transferable block. Crossover recombines whole sub-networks (with their weights) from two parents; mutation must re-roll the initialization lottery for a stuck component within one lineage. We count generations to solve all components (mean validation MSE < 0.08), over 60 search seeds, scanning the number of components at fixed per-component difficulty.

Table 2 is the resolution. With weights inherited, crossover wins again and by a wide margin: it solves 60/60 tasks in roughly twelve generations independent of the number of components, while mutation is 2.7 to 6.7× slower when it succeeds and unreliable when components accumulate (at $G = 3$ it solves only a third). The same recombination that did nothing to an architecture-only genome (Section 3.2) is decisive once the genome carries the solution, exactly as it was on the trap bits. This is the author’s intuition made concrete: a useful combination that evolution has found is inherited and keeps evolving, rather than being retrained from

Table 2: Weight-inheriting crossover versus mutation on the modular task (population 30, fine-tune 300 steps/generation, 60 search seeds, cap 120 generations). Crossover solves every task in about twelve generations whatever the component count; mutation is several times slower and, as components accumulate, frequently fails to solve at all within the cap.

components G	mutation (re-init)	crossover (inherit)
2	60/60, 31.0 gens	60/60, 11.4 gens
3	20/60 , 47.9 gens	60/60, 12.1 gens
4	59/60, 84.6 gens	60/60, 12.6 gens
5	59/60, 39.4 gens	60/60, 12.5 gens

zero, which is both what gives recombination a solution to transfer and what makes the search converge faster.

4 Discussion

The three experiments compose into one statement. Crossover’s building-block advantage is real and provable (Table 1), but it applies only when the genome carries a transferable solution. Standard neural architecture search does not: capacity is fungible and solutions live in weights that a retrain-from-scratch search discards, so recombination has nothing to work with, which is a structural reason random search is a strong NAS baseline and the field [9] routinely drops crossover. Restore the missing piece, weight inheritance, and the advantage returns in full (Table 2). The lesson is not that crossover is weak or strong in the abstract, but that it is exactly as useful as the genome is a carrier of solutions.

Limitations. Scale is small and tasks synthetic; the modular weight-inheritance result is a controlled probe with a fixed per-component task and a fixed number of modules, not yet the full self-modifying engine, which retrains each candidate and would need weight inheritance wired through its process-isolated worker model. Mean squared error stands in for classification error. None of this is hidden.

5 Conclusion and future work

Handed only the problem and a target error, the search discovers architecture, layer type, and training hyper-parameters. Recombination does not speed the architecture search while the genome carries only shape, and we now understand exactly why and exactly when it does: crossover needs building blocks that carry a solution, and weight inheritance supplies them (Table 2). The clear next step is engineering rather than science: carry trained weights through the engine’s worker model so the full self-modifying search inherits and recombines learned state, not only architecture. Two further steps remain: a two-dimensional convolution and an image task, where architecture is decisive rather than fungible; and a bridge to a standard tabular NAS benchmark.

Reproducibility

Every result is regenerated from [13] with a single command. The engine builds with `make` (C99, no dependencies) into `smbpann` (worker), `evolve` (search and control, with `-X` for crossover), and `gentask`; `make ut` runs the unit and gradient-check suite under AddressSanitizer and UB-San. The three validation probes are standalone: `make bbtest` (Table 1, building-block traps),

`make modnas` (the parallel-branch gradient check), and `make modevo` (Table 2, weight-inheriting crossover; run `./modevo G freq`). The neural crossover A/B is `scripts/xover_ab.sh`. Each run uses a fixed 32-bit xorshift generator, so the numbers are identical on re-running.

References

- [1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature* 323, 1986.
- [2] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks*. Seminar in Artificial Intelligence, Open University of Israel, 1997.
- [3] I. Rechenberg. *Evolutionstrategie*. 1973; H.-P. Schwefel, *Numerical Optimization of Computer Models*, 1981.
- [4] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975 (schemata and building blocks).
- [5] K. Deb, D. E. Goldberg. Analyzing Deception in Trap Functions. *Foundations of Genetic Algorithms* 2, 1993.
- [6] Y. LeCun, B. Boser, J. S. Denker, et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.
- [7] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.
- [8] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.
- [9] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [10] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
- [11] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.
- [12] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.
- [13] V. Gavrilov. SMBPANN: a self-modifying backpropagation feed-forward neural network. Source code, <https://github.com/Anode1/SMBPANN>.